A.R.T.E.M.I.S. v2.0

Autonomous Radar Thermal Engine Monitoring Illegal Smuggling

Prepared For: PROTONEX 2026 Hackathon
Role: Senior Systems Engineering Team
Document Version: 2.0

---

1. Abstract

This paper details the architecture, deployment, and performance metrics of A.R.T.E.M.I.S. (Autonomous Radar Thermal Engine Monitoring Illegal Smuggling), a low-latency, full-stack sentry defense system. Traditional perimeter security in remote terrains suffers from severe scalability and human-risk limitations. A.R.T.E.M.I.S. bridges edge-computing microcontrollers with a high-performance web dashboard via a custom Serial-to-WebSocket pipeline. By integrating an Arduino UNO for ultrasonic spatial mapping, an ESP-32 for real-time local telemetry, and an isolated ESP32-CAM for visual evidence logging, the system identifies physical incursions with a measured end-to-end latency of under 100 milliseconds.

2. Problem and Motivation

Illegal smuggling, unauthorized timber extraction, and wildlife poaching pose existential threats to global biodiversity and border integrity. These illicit operations exploit vast, remote topographies where continuous human surveillance is financially and structurally unfeasible.

Alignment with UN SDG-15 (Life on Land):
A.R.T.E.M.I.S. was engineered to directly address United Nations Sustainable Development Goal 15, which mandates the protection of terrestrial ecosystems and the halting of biodiversity loss. By deploying autonomous IoT sentries as a decentralized early-warning infrastructure, A.R.T.E.M.I.S. acts as a force multiplier for under-resourced forest rangers and border authorities. It automates the detection of smugglers and poachers, shifting the security paradigm from reactive damage control to proactive interception, thereby preserving fragile ecosystems.

3. Methodology

The system architecture follows a strict decoupled, modular engineering approach, divided into three operational layers to prevent processing bottlenecks:

3.1 Edge / Data Acquisition Layer

The primary spatial mapping is handled by a 5V Arduino UNO. An HC-SR04 Ultrasonic Sensor is mounted on an SG90 Micro Servo, executing a continuous 180-degree acoustic sweep. The Arduino acts strictly as a dumb-sensor node; it performs zero UI processing. Its sole function is to acquire range data and push a highly structured UART packet (e.g., `<THREAT:45,20>`) at 115200 baud to the bridging layer.

3.2 Bridging / Local Processing Layer

An ESP-32 standard microcontroller operates as the tactical core. It ingests the serial payload from the Arduino and drives a local 1.8" ST7735 TFT display. To achieve a cinematic, flicker-free radar visualization, the firmware utilizes the `TFT_eSPI` library and an off-screen sprite buffer. The radar graphics (including fading phosphor trails and expanding concentric waves) are drawn in the ESP-32's internal RAM and pushed to the display in a single DMA transaction. Simultaneously, an ESP32-CAM operates as a standalone optical node.

3.3 Command / Visualization Layer

The ESP-32 bridges the physical world to the web via a Node.js backend running the `SerialPort` library. The backend parses the UART stream and emits lightweight JSON payloads over `Socket.io`. The frontend is a React application utilizing Tailwind CSS. It synthesizes the telemetry into a high-performance military HUD, overlaying the ESP32-CAM's MJPEG stream with dynamic SVG crosshairs and utilizing the Web Audio API for zero-latency localized alarms.

4. Results and Engineering Solutions

The deployed prototype successfully validated the core hypothesis: an edge-to-cloud IoT sentry system can reliably automate perimeter security.

- **Latency Metrics:** Empirical testing yielded an end-to-end telemetry propagation latency (from physical object detection to React DOM update) of < 100ms.
- **Overcoming Logic Level Asymmetry:** A significant technical hurdle was bridging the 5V logic of the Arduino TX with the 3.3V logic of the ESP-32 RX2. This was mitigated by strictly enforcing a shared common ground, utilizing a high-speed 115200 baud rate, and implementing strict string-matching parsers (`<ANGLE,DISTANCE>`) to drop corrupted frames instantly.
- **Isolating Blocking I/O:** Initial designs integrated the camera onto the main ESP-32. However, writing JPEGs to an SD card is a highly blocking operation that stalled the radar sweep. By isolating the ESP32-CAM as a standalone node, the primary ultrasonic telemetry loop remains completely unimpeded, guaranteeing 100% uptime for threat detection.

5. Conclusion and Future Enhancements

A.R.T.E.M.I.S. effectively demonstrates how modular edge-computing, when paired with a highly optimized full-stack web architecture, can provide enterprise-grade perimeter security.

Moving forward, the system architecture allows for extensive enterprise upgrades:

- **Machine Learning Target Classification:** Deploying TensorFlow Lite (TinyML) directly onto the ESP32-CAM to autonomously differentiate between wildlife, humans, and vehicles, reducing false positives.
- **ESP-NOW Mesh Networking:** Transitioning from centralized Wi-Fi to a decentralized ESP-NOW topology. This will allow hundreds of A.R.T.E.M.I.S. nodes to daisy-chain telemetry across vast, off-grid forest sectors, creating an impenetrable digital border.